

DESIGN PATTERNS JUSTIFICATION & CLASS DIAGRAMS

E-commerce Platform



*Arjola Metaliaj, Ederta Kariqi, Asja Xhelezi, Stilianos Troka, Elda
Toska, Sindi Terihati, Elisa Spaho*

STATE PATTERN

What is the State Pattern?

The State Pattern is a behavioral design pattern that allows an object to change its behavior depending on its current state. Instead of using large conditional statements to manage different behaviors, the pattern separates each state into its own class.

The main object, called the context, stores a reference to a state object and delegates state-dependent behavior to it. As the state changes, the object behaves differently without modifying its own core structure.

This pattern improves maintainability, scalability, and readability by encapsulating state-specific logic into separate classes.

RELEVANCE TO OUR SYSTEM DESIGN

The State Pattern was selected because the order processing system naturally operates through a sequence of lifecycle states.

An order does not behave the same throughout its existence. Different operations become valid or invalid depending on the current order status.

For example:

- A pending order can be confirmed or cancelled.
- A confirmed order can be shipped.
- A shipped order can be delivered.
- A delivered order can only be refunded.

Because of this changing behavior, the State Pattern is highly suitable for modeling the order lifecycle.

Instead of implementing complex conditional statements such as:

```
if status == "Pending"
if status == "Confirmed"
if status == "Shipped"
```

The system delegates behavior to specialized state classes making for a more elegant and extensible architecture

.

DESIGN CHOICES MADE

1. Order as the Context Class

The Order class was designed as the context object because it is the entity whose behavior changes during the order lifecycle.

The class stores:

- order information,
- the current state object,
- and the high-level operations available to the system.

The Order object delegates state-dependent operations to the active OrderState.

2. Abstract OrderState Superclass

An abstract superclass named OrderState was used instead of an interface.

This decision was made because:

- the lecture slides model the pattern using inheritance,
- all states share common structure and behavior,
- and each state requires access to the owning order object.

The superclass also contains:

- a reference to the Order,
- shared operation definitions,
- and a constructor used by all concrete states.

This creates stronger consistency between states.

3. Concrete State Classes

Separate classes were created for:

- PendingState
- ConfirmedState
- ShippedState
- DeliveredState

Each class contains only the operations valid for that specific state.

For example:

- PendingState allows confirmation and cancellation.
- ConfirmedState allows shipping.
- DeliveredState allows refunds only.

This design prevents invalid operations and models real business rules more accurately.

DESIGN CHOICES MADE

4. Aggregation Relationship

An aggregation relationship was used between Order and OrderState. This was chosen because:

- the order maintains a reference to a current state object,
- but the state logic remains separated from the order itself,
- supporting loose coupling and delegated behavior.

5. Lifecycle-Focused Operations

The operations included in the diagram were intentionally focused on lifecycle behavior:

- `confirmOrder()`
- `cancelOrder()`
- `shipOrder()`
- `deliverOrder()`
- `refundOrder()`

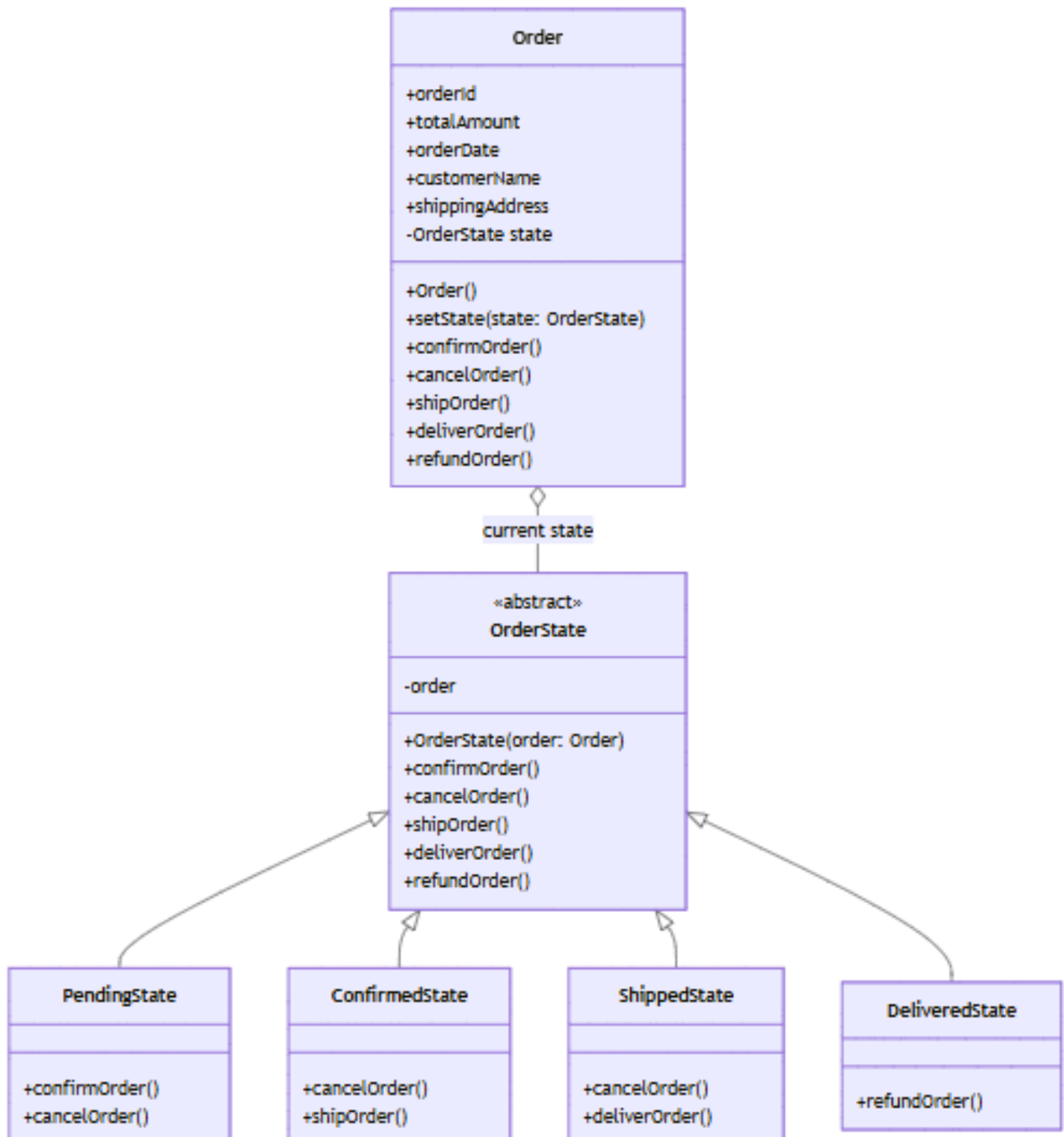
These operations directly represent transitions and actions affected by state changes, which aligns closely with the purpose of the State Pattern.

BENEFITS OF THIS DESIGN

Using the State Pattern provides several advantages in the system:

- Eliminates large conditional statements.
- Encapsulates state-specific behavior.
- Makes the order lifecycle easier to maintain.
- Simplifies adding new states in the future.
- Improves readability and scalability.
- Models real-world order processing behavior more accurately.

CLASS DIAGRAM



FAÇADE PATTERN

What is the Façade Pattern?

The Façade Pattern is a structural design pattern that provides a simplified interface to a complex subsystem made up of many classes and operations.

Instead of the client interacting directly with multiple services, the client communicates only with the façade, which internally coordinates the workflow between subsystem components.

The pattern helps reduce complexity, hide implementation details, and make the system easier to use and maintain.

RELEVANCE TO OUR SYSTEM DESIGN

The Façade Pattern was selected because the checkout process in the e-commerce platform naturally involves multiple independent services that must work together in a specific sequence.

During checkout:

- payment must be processed,
- stock must be validated and reserved,
- shipment information must be created,
- invoices must be generated,
- and confirmation notifications must be sent.

Without a façade, the client would need to communicate with every service individually, creating tightly coupled and difficult-to-maintain code.

Because of this complex interaction between services, the Façade Pattern is highly suitable for simplifying the checkout workflow.

Instead of implementing complicated coordination logic inside the frontend or controller classes, the system delegates the orchestration responsibility to a dedicated façade class.

DESIGN CHOICES MADE

1. CheckoutPage as the Client

The CheckoutPage class was designed as the client because it represents the component initiating the checkout process.

The page does not directly communicate with all subsystem services. Instead, it interacts only with:

- ICheckoutFacade

through the operation:

- checkout(order : Order)

This keeps the frontend simpler and reduces subsystem dependency exposure.

2. ICheckoutFacade Interface

An interface named ICheckoutFacade was used to define the checkout operation.

This decision was made because:

- it improves abstraction,
- reduces coupling,
- and allows flexibility for future implementations.

The client depends on the interface rather than the concrete façade implementation.

3. CheckoutFacade as the Central Coordinator

CheckoutFacade was designed as the main façade class because it coordinates the entire checkout workflow.

The class internally manages:

- payment processing,
- stock validation and reservation,
- shipment creation,
- invoice generation,
- and notification delivery.

The façade hides the complexity of subsystem communication and exposes only a single simplified operation to the client.

DESIGN CHOICES MADE

4. Specialized Service Classes

Separate subsystem classes were created for:

- PaymentService
- InventoryService
- ShippingService
- InvoiceService
- NotificationService

Each class contains only the operations related to its specific responsibility.

For example:

- PaymentService handles payment processing.
- InventoryService validates and reserves stock.
- ShippingService creates shipment information.

This design improves modularity and separates responsibilities clearly.

BENEFITS OF THIS DESIGN

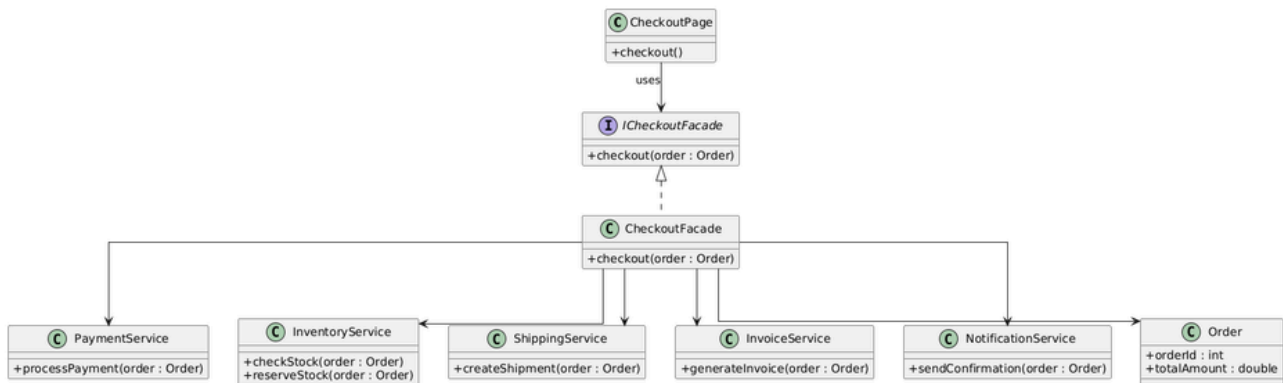
Using the Façade Pattern provides several advantages in the system:

- Reduces complexity exposed to the client.
- Simplifies the checkout workflow.
- Centralizes orchestration logic.
- Improves maintainability and readability.
- Reduces coupling between components.
- Makes future extensions easier.
- Hides subsystem implementation details from the client.

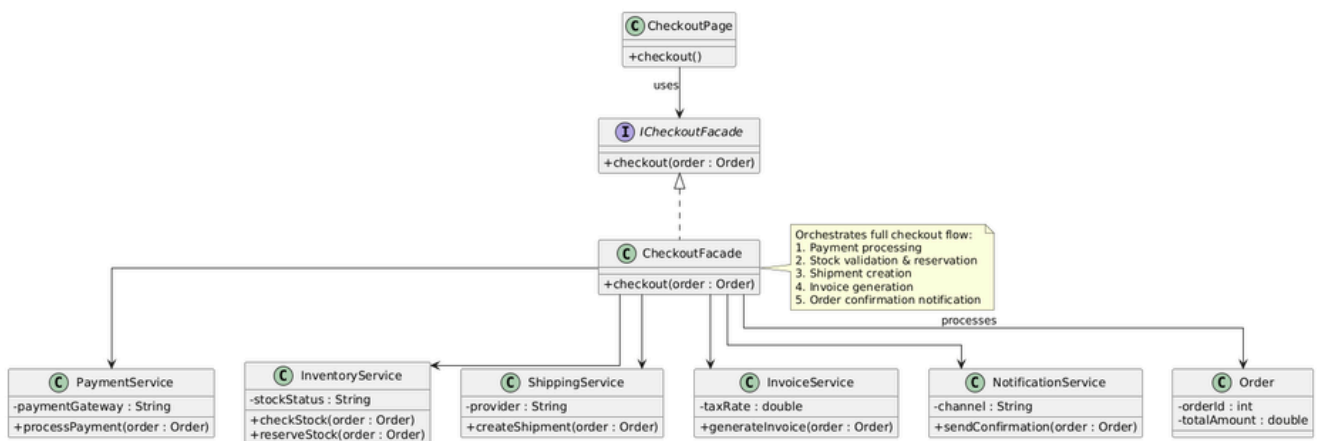
One possible drawback is that the façade could eventually become too large and overloaded with responsibilities if too much functionality is added to it.

However, when kept focused on workflow coordination, the Façade Pattern creates a cleaner and more scalable architecture for the checkout system.

CLASS DIAGRAM



CLASS DIAGRAM WITH ATTRIBUTES



CHAIN OF RESPONSIBILITY PATTERN

What the Chain of Responsibility Pattern Is

The Chain of Responsibility Pattern is a behavioral design pattern that allows a request to pass through a sequence of handler objects until all required processing steps are completed.

Instead of placing all processing logic inside one large method, each handler is responsible for performing a single task and forwarding the request to the next handler in the chain.

This pattern improves modularity, flexibility, and maintainability by separating responsibilities into independent processing components.

RELEVANCE TO OUR SYSTEM DESIGN

The Chain of Responsibility Pattern was selected because the order processing system performs multiple sequential validation steps before an order can be completed.

An order request must pass through several checks, such as:

- user authentication,
- cart validation,
- stock availability verification,
- and payment validation.

Each step represents a separate responsibility within the checkout workflow.

Instead of implementing all validations inside one complex method with nested conditions, the request is passed through a chain of specialized handlers.

This creates a cleaner and more scalable processing architecture.

DESIGN CHOICES MADE

1. Abstract OrderHandler Superclass

An abstract superclass named OrderHandler was created to represent a generic request handler.

The superclass defines:

- the `handleOrder()` operation,
- and the mechanism for forwarding requests through the chain.

This follows the exact handler-chain structure presented in the lecture slides.

DESIGN CHOICES MADE

2. Specialized Validation Handlers

Separate handler classes were created for:

- AuthenticationHandler
- CartValidationHandler
- StockValidationHandler
- PaymentValidationHandler

Each handler performs only one validation task before forwarding the order request to the next handler. This follows the Single Responsibility Principle and keeps the validation workflow modular.

3. Sequential Request Processing

The request moves through the validation chain step-by-step.

For example:

1. User authentication is verified.
2. Cart contents are validated.
3. Product stock is checked.
4. Payment information is validated.

If any validation fails, the processing sequence stops immediately. This creates an efficient and controlled order validation pipeline.

4. Loose Coupling Between Components

The sender communicates only with the first handler in the chain.

The client does not need knowledge about:

- how many handlers exist,
- their internal implementation,
- or the processing order.

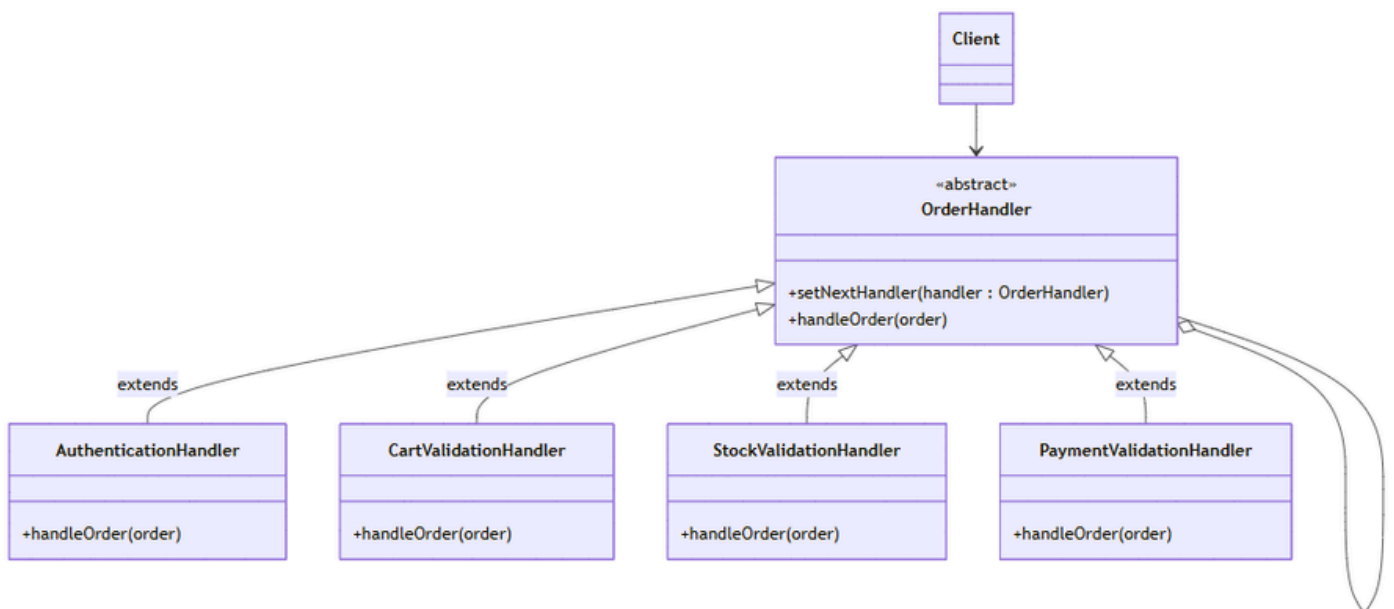
This reduces coupling and improves system flexibility.

BENEFITS OF THIS DESIGN

Using the Chain of Responsibility Pattern provides several advantages:

- Eliminates large validation methods.
- Separates processing responsibilities into independent components.
- Simplifies maintenance and scalability.
- Makes validation flow easier to extend.
- Supports flexible checkout workflows.
- Closely matches real-world enterprise processing pipelines.

CLASS DIAGRAM



ADAPTER PATTERN

What the Adapter Pattern Is

The Adapter Pattern is a structural design pattern that allows objects with incompatible interfaces to collaborate by converting one interface into another that the system expects.

Instead of modifying existing classes or external services directly, the pattern introduces an intermediate adapter object that translates requests and responses between components. The pattern helps improve compatibility, reduce coupling, and simplify integration with external systems while preserving existing implementations.

RELEVANCE TO OUR SYSTEM DESIGN

The Adapter Pattern was selected because the e-commerce platform interacts with multiple delivery and shipment providers that may expose tracking information in different formats.

During delivery tracking:

- tracking information must be requested,
- responses must be standardized,
- provider-specific formats must be converted,
- customers must receive consistent tracking updates.

Without an adapter, the system would need separate logic for every logistics provider, creating tightly coupled and difficult-to-maintain code. Because of this variation between external service interfaces, the Adapter Pattern is highly suitable for simplifying logistics integration. Instead of implementing provider-specific conversion logic inside the frontend or tracking module, the system delegates format conversion to dedicated adapter classes.

DESIGN CHOICES MADE

1. TrackingPage as the Client

The TrackingPage class is designed as the client because it represents the component requesting delivery status information. The page doesn't communicate directly with logistics providers, but it only interacts with:

- ITrackingService
- through the operation:
- trackOrder(orderId : int)

This keeps the frontend independent from external provider implementations.

DESIGN CHOICES MADE

2. ITrackingService Interface

An interface named ITrackingService was used to define tracking operations.

This decision was made because:

- it improves abstraction,
- reduces dependency on specific providers,
- allows integration of new logistics services in the future.

The client depends on the interface rather than concrete implementations.

3. TrackingAdapter as the Translator

TrackingAdapter was designed as the main adapter class because it converts external tracking data into a unified system format.

The adapter internally manages:

- request conversion,
- provider communication,
- response translation,
- standardized tracking output.

The adapter hides incompatibilities between systems and exposes only a single tracking interface.

4. Specialized Logistics Provider Classes

Separate provider classes were created for:

- DHLTrackingService
- FedExTrackingService
- LocalCourierService
- Each provider contains only operations related to communication with that specific provider.

For example:

- DHLTrackingService retrieves DHL tracking data.
- FedExTrackingService retrieves FedEx shipment information.
- LocalCourierService retrieves local delivery status.

This design improves modularity and separates external integrations clearly.

BENEFITS OF THIS DESIGN

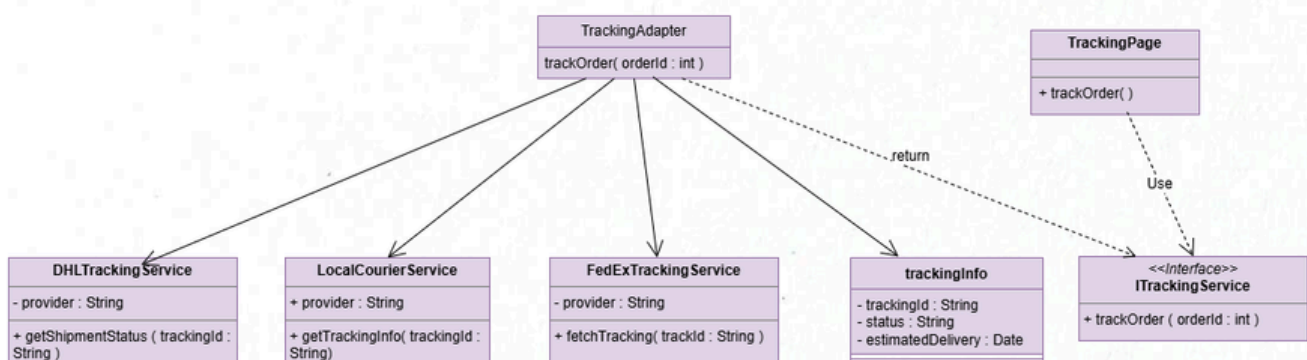
Using the Adapter Pattern provides several advantages in the system:

- Reduces dependency on external APIs.
- Simplifies logistics integration.
- Standardizes tracking information.
- Improves maintainability and readability.
- Makes future provider additions easier.
- Hides external implementation differences.

One possible drawback is that additional adapter classes increase system complexity when many providers are introduced.

However, when focused on interface conversion, the Adapter Pattern creates a cleaner and more extensible architecture for delivery tracking.

CLASS DIAGRAM



STRATEGY PATTERN

What is the Strategy Pattern?

The Strategy Pattern is a behavioral design pattern that allows a system to dynamically change the algorithm or behavior used to perform a specific task at runtime.

Instead of implementing all recommendation logic inside a single class using large conditional statements, the Strategy Pattern separates each recommendation algorithm into its own strategy class.

The main object, called the context, stores a reference to a recommendation strategy object and delegates recommendation generation to it.

This allows the system to switch recommendation methods depending on customer behavior, browsing activity, or business requirements without modifying the core recommendation system structure.

The pattern improves:

- maintainability
- scalability
- flexibility
- and separation of concerns.

RELEVANCE TO OUR SYSTEM DESIGN

The Strategy Pattern was selected because the e-commerce platform supports multiple personalized recommendation approaches.

According to the system requirements and use cases, customers can receive:

- personalized product recommendations
- browsing-based suggestions
- purchase-history recommendations
- and trending product suggestions.

Different recommendation techniques may be more suitable depending on:

- customer activity
- available data
- or platform goals.

RELEVANCE TO OUR SYSTEM DESIGN

For example:

- A new customer may receive trending products.
- A returning customer may receive purchase-history recommendations.
- A highly active customer may receive browsing-based recommendations.

Instead of implementing complex conditional statements such as:

```
if customer.isNew()
```

```
if customer.hasPurchaseHistory()
```

```
if customer.hasRecentBrowsing()
```

the system delegates recommendation generation to specialized strategy classes, creating a more extensible and maintainable architecture.

This design aligns closely with:

- UC-25 Personalized Recommendations
- UC-43 Personalized Product Recommendation

DESIGN CHOICES

1. RecommendationEngine as the Context Class

The RecommendationEngine class was designed as the context object because it is responsible for generating product recommendations for customers.

The class stores:

- customer information
- the currently selected recommendation strategy
- and the high-level recommendation operations available to the system.

The RecommendationEngine delegates recommendation generation behavior to the active RecommendationStrategy object.

2. RecommendationStrategy Interface

An interface named RecommendationStrategy was used to define the common recommendation operation shared by all recommendation algorithms.

DESIGN CHOICES

This decision was made because:

- all recommendation strategies perform the same high-level task
- strategies can be replaced dynamically
- and each recommendation method requires different internal logic.

The interface contains:

- a shared `recommend(customerId)` operation,

allowing all concrete strategies to follow a consistent structure.

This improves flexibility and supports runtime strategy switching.

3. Concrete Strategy Classes

Separate strategy classes were created for different recommendation approaches:

- `BrowsingHistoryStrategy`
- `PurchaseHistoryStrategy`
- `TrendingProductsStrategy`

Each class contains only the logic required for its specific recommendation algorithm.

For example:

- `BrowsingHistoryStrategy` recommends products based on recently viewed categories.
- `PurchaseHistoryStrategy` recommends products related to previous purchases.
- `TrendingProductsStrategy` recommends globally popular products for new or inactive users.

This design isolates recommendation algorithms and prevents recommendation logic from becoming centralized inside a single large class.

4. Aggregation Relationship

An aggregation relationship was used between `RecommendationEngine` and `RecommendationStrategy`.

This was chosen because:

- the engine maintains a reference to the current strategy object
- recommendation logic remains separated from the engine itself
- and strategies can be changed without modifying the `RecommendationEngine` class.

This supports loose coupling and delegated behavior.

DESIGN CHOICES

5. Recommendation-Focused Operations

The operations included in the diagram were intentionally focused on recommendation generation behavior:

- `setStrategy()`
- `generateRecommendations()`

These operations directly represent the responsibilities of the Strategy Pattern:

- selecting a recommendation algorithm,
- and executing recommendation generation dynamically.

This closely matches the platform's personalized recommendation workflow.

BENEFITS OF THIS DESIGN

Using the Strategy Pattern provides several advantages in the system:

- Eliminates large conditional statements.
- Encapsulates recommendation algorithms into separate classes.
- Makes recommendation logic easier to maintain.
- Supports adding new recommendation methods in the future.
- Improves scalability and flexibility.
- Allows runtime switching between recommendation algorithms.
- Models real-world personalization behavior more accurately.

CLASS DIAGRAM

